

Problem Set 2 Solutions

Problem 1

Given the equation of motion

$$\dot{x} = x^3 + xu, \quad (1)$$

and the scalar function

$$V(x) = 2x^2 \quad (2)$$

$$\frac{\partial V}{\partial x} = 4x \quad (3)$$

and

$$u(x) = -4x^2. \quad (4)$$

a (3 points)

$$\dot{V} = 4x\dot{x} \quad (5)$$

$$= 4x(x^3 + x(-4x^2)) \quad (6)$$

$$= 4x(-3x^3) \quad (7)$$

$$= -12x^4 \quad (8)$$

$\dot{V} < 0$ and $\dot{V} \leq 0$, the system is GAS and globally stable isL. It is not exponentially stable because there is no positive constant α such that $\dot{V} < -\alpha V(x)$.

b (4 points) Let $J(x) = V(x)$

We know:

$$0 = \min_u \left\{ g(x, u) + \frac{\partial J^*}{\partial x} f(x, u) + \frac{\partial J^*}{\partial t} \right\} \quad (9)$$

Plugging in, we get:

$$0 = \min_u \{ g(x, u) + 4x(x^3 + xu) \} \quad (10)$$

$$\frac{\partial g}{\partial u} + 4x^2 = 0 \quad (11)$$

Therefore we can see that if we have a quadratic cost on u , e.g. $u^T R u$,

$$2ru = 4x^2 \quad (12)$$

$$u = -4 \frac{x^2}{2r} \quad (13)$$

u will minimize the HJB if $R = \frac{1}{2}$. Plugging back in we have

$$0 = g(x) + \frac{1}{2}u^{*2} + 4x(x^3 + xu^*) \quad (14)$$

$$0 = g(x) + \frac{1}{2}(-4x^2)^2 - 12x^4 \quad (15)$$

$$0 = g(x) + 8x^4 - 12x^4 \quad (16)$$

$$0 = g(x) - 4x^4 \quad (17)$$

$$g(x) = 4x^4 \quad (18)$$

Therefore $g(x, u) = 4x^4 + \frac{1}{2}u^2$

Problem 2

a (2 points)

The vector field for the problem defined in this system can be seen in figure 1. Intuitively, this system is a reversed

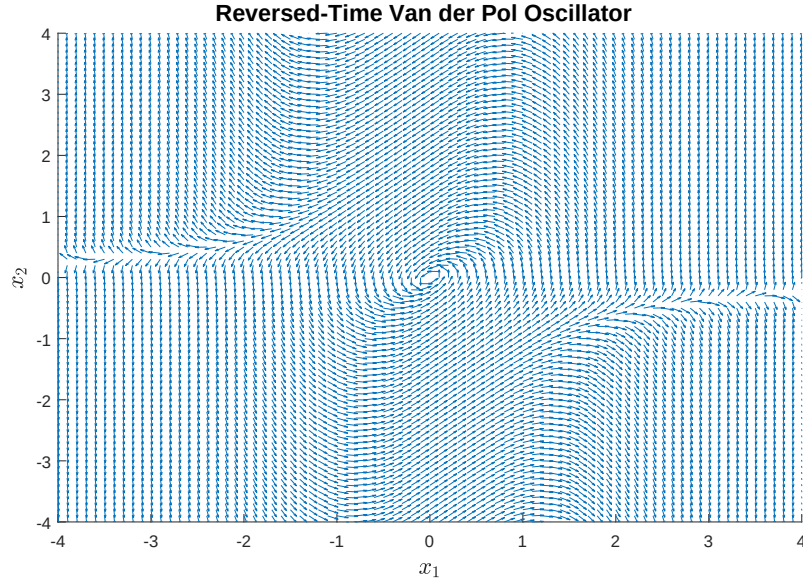


Figure 1: Van der Pol Oscillator Vector Field

time version of the Van der Pol Oscillator. Instead of all trajectories converging to a limit cycle, all trajectories converge to the origin within a particular region of state space and go to infinity outside of that region. This feature makes this nonlinear system well suited for testing ROA algorithms.

b (2 points)

The linearized system about the origin can be written as:

$$\dot{\bar{x}} = A\bar{x} \quad (19)$$

where

$$A = \begin{bmatrix} 0 & -1 \\ 1 & -1 \end{bmatrix} \quad (20)$$

and $\bar{x} = x - x_0$.

To find P, we use the following MATLAB code:

```
Q=eye(2)
P=lyap(A',Q);
```

We find that

$$P = \begin{bmatrix} 1.5 & -0.5 \\ -0.5 & 1.0 \end{bmatrix} \quad (21)$$

c (4 points)

We know that $V = x^T P x$ and that $\dot{V} = \frac{\partial V}{\partial x} \dot{x}$, which is equivalent to $\dot{V} = 2x^T P f(x)$. Therefore, for our system,

$$\dot{V} = -x_1^3 x_2 - 2x_1^2 x_2^2 - x_1^2 - 4x_1 x_2 + 3x_2^2 \quad (22)$$

To find a RoA via sampling, we start by examining points within the level set of the Lyapunov function $x^T P x < \rho$ for a small value of ρ . If $\dot{V} < 0$, we continue to increase ρ until $\dot{V}$ becomes positive, at which point we stop increasing ρ . The level set represented by our final value of ρ is the largest region of attraction we are able to find and is shown in figure 2.

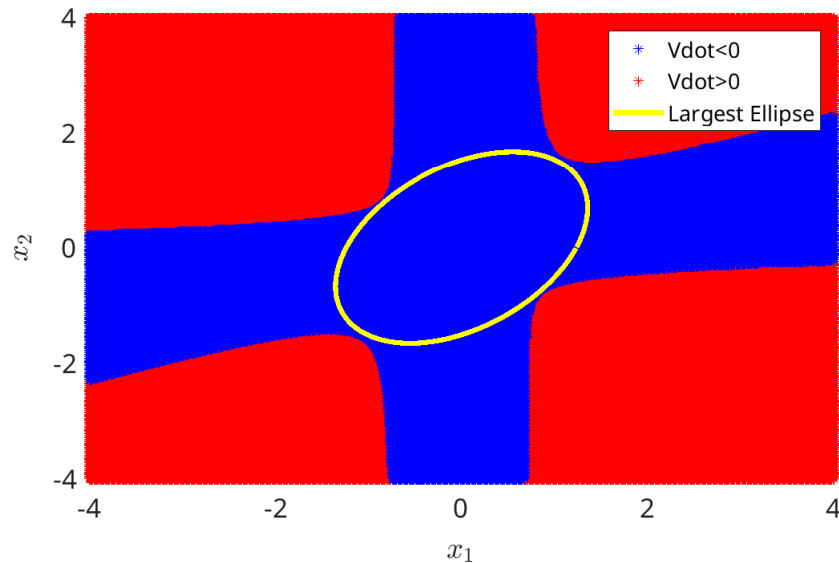


Figure 2: Van der Pol Oscillator Sample-Based Search

d (5 points)

Finally, we implement a SOS algorithm as follows:

```
x=msspoly('x',2);
c=msspoly('c',6);
r=2.301;
g=msspoly('g');

f(1,:) = -x(2);
f(2,:) = x(1)+(x(1)^2-1)*x(2);

P=[1.5 -0.5;-0.5 1];

Vdot=f'*P*x+x'*P*f;
```

```

V=x'*P*x;

s=c(1)*x(1)^2+c(2)*x(1)*x(2)+c(3)*x(2)^2+c(4)*x(1)+c(5)*x(2)+c(6)

q=s*(V-r)-Vdot-g*(x'*x);

pr=mssprog;
pr.free=c;
pr.free=g;
pr.sos=q;

pr.sedumi=-g;

```

For this algorithm, we must increase r until g is no longer positive. The results of this SOS program can be found in 3.

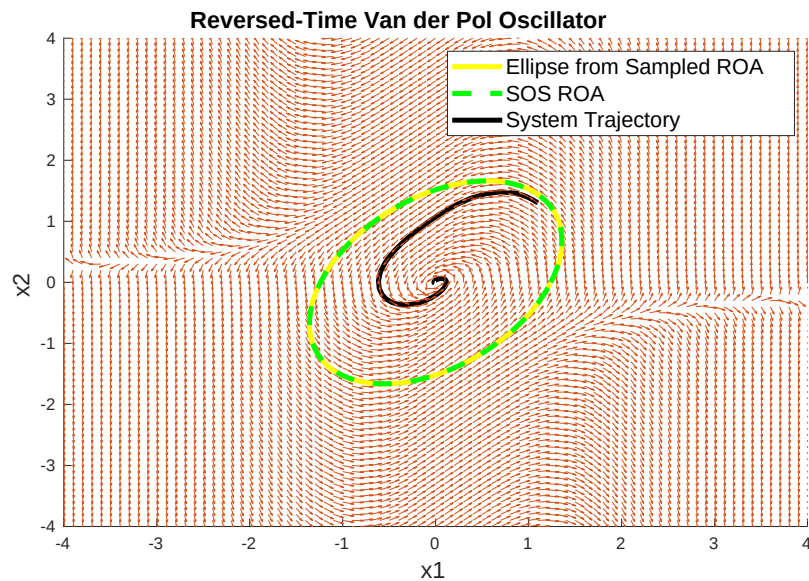


Figure 3: Van der Pol Oscillator SOS ROA

e (2 points)

In general, a region of attraction found from sampling is slightly larger than a region found using SOS. However, for this simple example, both regions are about the same.

f (2 points)

See figure 3.

Problem 3

a (1 point)

```

%=====
% gradients for the polynomial dynamics
%=====
function [dfdx, dfdu] = gradients(x)
    dfdx = [-1 0 1;

```

```

2*x(1)-2*x(3) -1 -2*x(1)+1;
0 -1 0];

dfdu = [0;0;1];
end

```

b (1 point)

$$\mathbf{B} = [0, 0, 1]^T \quad (23)$$

$$\mathbf{b}_1 = 0, \quad \mathbf{b}_2 = 0, \quad \mathbf{b}_3 = 1 \quad (24)$$

$$\frac{\partial \mathbf{b}_i}{\partial \mathbf{x}} = \mathbf{0} \quad \forall i \quad (25)$$

$$\frac{\partial \mathbf{W}}{\partial \mathbf{x}} \mathbf{b}_i - \frac{\partial \mathbf{b}_i}{\partial \mathbf{x}} \mathbf{W} - \mathbf{W} \frac{\partial \mathbf{b}_i}{\partial \mathbf{x}}^T \quad (26)$$

$$= 0 \quad \text{for } i = 1 \quad (27)$$

$$= 0 \quad \text{for } i = 2 \quad (28)$$

$$= \frac{\partial \mathbf{W}}{\partial \mathbf{x}} = 0 \quad \text{for } i = 3 \quad (29)$$

Thus,

$$\frac{\partial \mathbf{W}}{\partial \mathbf{x}} \mathbf{b}_i - \frac{\partial \mathbf{b}_i}{\partial \mathbf{x}} \mathbf{W} - \mathbf{W} \frac{\partial \mathbf{b}_i}{\partial \mathbf{x}}^T = 0 \quad \forall i \quad (30)$$

c (2 point)

```

%construct dual metric
W = W0 + W1*x(1) + W2*x(1)^2 + W3*x(2) + W4*x(2)^2 + W5*(x(1)*x(2));

%construct multiplier
rho0 = rho(1)+ rho(2)*x(1)+rho(3)*x(1)^2;

%compute Wdot
Wdot = (W1+2*W2*x(1) + W5*x(2))*f(1) + (W3+2*W4*x(2) + W5*x(1))*f(2);

```

d (3 point)

```

% contraction condition
P = -(-Wdot + A*W + W*A'-rho0*(B*B') + 2*lambda*W);

```

Set the constraint such that $y^T P y$ is SOS. Then, P must be positive definite

e (5 point)

```

optionsY = sdpsettings('solver','sedumi','verbose',1,'debug',1);

% setup sos constraint
F = [sos(y'*W*y), sos(rho0), sos(y'*P*y), W0==inv(S)];
solvesos(F, 0,optionsY, c);

```

```

%extract values for rho
rho_hat = value(rho);

%extract values for ccm matrices
What{1} = value(W0);
What{2} = value(W1);
What{3} = value(W2);
What{4} = value(W3);
What{5} = value(W4);
What{6} = value(W5);

```

f (2 point)

```

%=====
% cost function for computing the geodesic
%=====
function e = cost_function(X,W0)

N = (length(X)-1)/6;
x = reshape(X(1:N*3),3,N);
dxds = reshape(X(N*3+1:end-1),3,N);
ds = X(end);
e = 0;
    for i=1:N-1
        xi=x(:,i);
        dxdsi = dxds(:,i);
        Mx=evalMetric(W0,xi);
        e = e + dxdsi'*Mx*dxdsi*ds;
    end
e=e/1e6;
end

```

```

lb(:,1) = xi;
ub(:,1) = xi;
lb(:,N)=xd;
ub(:,N)=xd;

```

g (3 point)

```

%=====
% integrate deltaK
%=====
function K = integrateDeltaK(xs,W0,rho0,B,dxds,ds)
K= 0;
    for i=1:length(xs)
        Wx = evalDualMetric(W0,xs(:,i));
        rhox= evalRho(rho0,xs(:,i));
        K = K + evalDeltaK(rhox,B,Wx)*dxds(:,i)*ds;
    end
end

```

```

%=====
% evaluate delta K
%=====
function dK = evalDeltaK(rho,B,W)
    dK = -0.5*rho*B'*inv(W);
end

```

h (3 point)

Any initial conditions and plots that show CCM stabilizes but LQR doesn't is correct.

Problem 4

a (3 points)

$$J(x) = 2x_1^2 - 8x_1 - 4x_2 + 4x_2^2 + 12 \quad (31)$$

$$f(x) = x_1 + x_2 - 2 = 0 \quad (32)$$

$$(33)$$

$$L(x) = 2x_1^2 - 8x_1 - 4x_2 + 4x_2^2 + 12 - \lambda(x_1 + x_2 - 2) \quad (34)$$

$$\frac{\partial L}{\partial x_1} = 4x_1 - 8 - \lambda = 0 \quad (35)$$

$$\frac{\partial L}{\partial x_2} = -4 + 8x_2 - \lambda = 0 \quad (36)$$

$$\frac{\partial L}{\partial \lambda} = -x_1 - x_2 + 2 = 0 \quad (37)$$

$$(38)$$

$$x_1 = -x_1 + 2 \quad (\text{from eq. 7}) \quad (39)$$

$$0 = 4(-x_2 + 2) - 8 - \lambda \quad (\text{from eq. 5}) \quad (40)$$

$$\lambda = -4x_2 \quad (41)$$

$$0 = -4 + 8x_2 + 4x_2 \quad (\text{from eq. 6}) \quad (42)$$

$$x_2 = \frac{1}{3} \quad (43)$$

$$x_1 = \frac{5}{3} \quad (\text{from eq. 7}) \quad (44)$$

b (3 points)

```

function lagrangemultipliers
clear all
close all

xs=[-8:.1:8];
[x1 x2]=meshgrid(xs);

J=2*x1.^2-8*x1-4*x2+4*x2.^2+12;

c=2-xs;

```

```

figure()
hold on
contour(x1,x2,J)
h1=plot(xs,c,'r')
h2=plot([1+2/3 1+2/3],[1/3 1/3],'*')
hold off
legend([h1 h2],{'Constraint x1+x2=2 ','Optimal Point '},'FontSize',13);
xlabel('x1','FontSize',14);
ylabel('x2','FontSize',14);
title('Contour Plot of J(x)','FontSize',14);
set(gcf, 'PaperPosition', [0 0 8 5]);%Position plot at left hand corner
    with width 5 and height 5.
set(gcf, 'PaperSize', [8 5]); %Set the paper to have width 5 and height 5.
saveas(gcf, 'lagrange', 'pdf') %Save figure

x=[1+2/3 1/3];

J=2*x(1).^2-8*x(1)-4*x(2)+4*x(2).^2+12

end

```

Problem 5

a (2 points)

Indirect trajectory optimization methods use calculus of variations or the maximum principle to solve the trajectory optimization problem numerically. Direct methods directly transform an optimal control problem into a nonlinear program.

In a shooting approach, we satisfy the dynamics constraints via forward integration. Transcription approaches incorporate dynamic feasibility as an explicit constraint in the optimization problem formulation. Collocation methods represent the time-varying trajectories as splines.

b (5 points)

```

% =====
% Cartpole Backpropagation Through Time Trajectory Optimization
% =====
function cartpole_bptt()

close all
clear all

% set up the dynamics
addpath("dynamical_systems/")
param=init_cartpole_params();
GRADIENTS=@cartpole_grads;
DYNAMICS=@cartpole_dynamics;
DRAW=@draw_cartpole;
rmpath("dynamical_systems/")

param.dt=1/20; %time step

```



```

param.x0=[0 0 0 0]'; % initial conditions
param.xd = [0 pi 0 0]'; % final desired state
param.iter=200; % number of iterations
param.Q=diag([0 0 0 0]); % Cost on glider trajectory
param.R=.1; % actuator cost

param.Qend=diag([2000 2000 10 10]); % final value cost
param.eta=0.005; % optimization update parameter
param.x0=[0 0 0 0]';

%bounds for plotting
param.xL=-2;
param.xU=2;
param.yL=-2*pi;
param.yU=2*pi;

dt=param.dt;%time step

N=50;%number of time steps
param.N=N;

utape0=0*ones(1,N);%initialize the initial open loop policy

ti=0;
t=ti:dt:(N-1)*dt;%time values

x0=param.x0;%set the initial condition

utape=bptt(utape0,x0,DYNAMICS,GRADIENTS,param);% run the bptt algorithm

%simulate the dynamics
x=x0;
for k=1:N
    xtape(:,k)=x;
    xdot=DYNAMICS(t,x,utape(k),param);
    x=x+xdot*dt;
end

%plot the resulting trajectory
figure()
plot(xtape(1,:),xtape(2,:))
axis([param.xL param.xU param.yL param.yU])
figure()
plot(utape);

% draw the robot
for i=1:size(xtape,2)
    DRAW((i-1)*dt,xtape(:,i),param);
end

end

```

```

% =====
% This trajectory optimization method using back propagation through time
% to calculate the gradients and gradient descent to optimize the
% trajectory
% =====
function utape = bptt(utape0,x,DYNAMICS,GRADIENTS,param)
    utape = gradient_descent(utape0,x,DYNAMICS,GRADIENTS,param);
end

% =====
% This function implements gradient descent optimization
% =====
function utape = gradient_descent(utape0,x,DYNAMICS,GRADIENTS,param)

x0=x; % initial state position
N=param.N; % number of discrete time steps

J_old=100000; % initial cost
utape_old=utape0; % initial utape
utape=utape0;
eta=param.eta; % gradient descent optimization gain

    for k=1:param.iter

        % compute cost function gradients
        [J,dJdalpha] = bptt_gradients(utape0,x0,DYNAMICS,GRADIENTS,param);

        % compare current cost to previous cost

        if (J > J_old) % if current cost is greater, reduce optimization gain

            J = J_old;
            utape = utape_old;
            eta = eta/2;

        else % if current cost is less, increase the optimization gain
            eta = 1.1*eta;
        end

        % save last utape and cost
        utape_old = utape;
        J_old = J;

        % update input tape through gradient descent
        utape=utape0-eta*dJdalpha';

        utape0=utape;

    end
end

```

```

% =====
% This function uses backprop to compute the gradients
% =====

function [J,dJdalpha] = bptt_gradients(alpha_in,x0,DYNAMICS,GRADIENTS,
    param)

dt=param.dt; % discrete time interval
xd=param.xd; % final goal state

alpha = alpha_in; % input tape

% construct time vector
ti=0;
N=length(alpha);
T=ti:dt:(N-1)*dt;
tf=T(N);
utape=alpha_in;

x=x0;
for k=1:N
    xtape(:,k)=x;
    xdot=DYNAMICS(0,x,utape(k),param);
    x=x+xdot*dt;
end

% plot first state against second state
figure(24)
plot(xtape(1,:),xtape(2,:),xd(1),xd(2),'o',x0(1),x0(2),'*');
xlabel('x-coordinate (m)');
ylabel('z-coordinate (m)');
title('Real-time Planning');
axis([param.xL param.xU param.yL param.yU]);drawnow;

% compute the gradients of the cost function
dJdalpha = compute_gradients(xtape,utape,N,param,GRADIENTS);

% compute the cost
J=cost(xtape,utape,param) + finalCost(xtape(:,N),param);

end

% =====
% This function returns the gradients by
% integrating the adjoint equations
% =====
function dJdalpha = compute_gradients(x,u,N,param,GRADIENTS)

dt=param.dt;
Q=param.Q;
R=param.R;
Qend=param.Qend;
xd=param.xd;

```

```

Q = Q.*dt; R = R.*dt;

[dfdx,dfdu] = GRADIENTS(0,x(:,N),u(:,N),param);

dgdu = 2*u(:,N)'*R;
dudalpha = 0*u; dudalpha(:,N)=1; %gradient of u w.r.t. parameters for open
    loop policy at N

F_alpha = dfdu*dudalpha;
G_alpha = dgdu*dudalpha;

y = -Qend*(x(:,N)-xd); %give terminal condition for y

dJdalpha = (G_alpha'-F_alpha'*y).*dt; % dJdalpha for first time step

for n=N-1:-1:1 %integrate adjoint equations backwards in time

    dgdx = x(:,n)'*Q; %gradient of cost with respect to current state
    dgdu = u(:,n)'*R; %gradient of cost with respect to current action

    [dfdx,dfdu] = GRADIENTS(0,x(:,n),u(:,n),param); %gradient of f w.r.t.
        current position, action
    F_x = dfdx;
    G_x = dgdx;
    dudalpha = 0*dudalpha(:,1:N); dudalpha(:,n) = 1; %gradient of u w.r.t.
        parameters for open loop policy at current time
    F_alpha = dfdu*dudalpha;

    G_alpha = dgdu*dudalpha;

    y = inv(eye(length(F_x))-F_x'*dt)*(y-G_x'*dt); %solve for y

    dJdalpha = dJdalpha+(G_alpha'-F_alpha'*y).*dt; %add this step's
        contribution to dJdalpha

end

end

% =====
% This function defines the instantaneous cost (i.e. g(x,u))
% =====
function J = cost(X,u,param)

R=param.R;
Q=param.Q;
dt=param.dt;

R=R*dt;
Q=Q*dt;

J=0;

```

```

for k=1:length(X)
    J=J+X(:,k)'*Q*X(:,k)+R*u(k)^2;
end

end

% =====
% Implements a final cost
% =====
function Jf = finalCost(X,param)

xd=param.xd;
Qend=param.Qend;

Xerr = X - repmat(xd,1,size(X,2));

Jf = Xerr'*Qend*Xerr;

end

```

c (5 points)

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Function computes optimal swing-up trajectory via direct transcription
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function cartpole_dirtran

clear all
close all

%set up the dynamics
addpath("dynamical_systems/")
param=init_cartpole_params();
GRADIENTS=@cartpole_grads;
DYNAMICS=@cartpole_dynamics;
DRAW=@draw_cartpole;
addpath("dynamical_systems/")

N = 50;% number of timesteps
param.dt=1/20; %discrete timestep

%set up the optimization parameters
param=opt_param(param);
param.nX=4; %state size
param.nU=1; %input size
param.xd=[0 pi]; %desired final state

param.plot=0;
tic
% generate a trajectory via direct transcription
[w,F,info] = dirtran(N,DYNAMICS,GRADIENTS,param);
toc

```

```

info

% convert optimization parameters into state and control trajectories
xtape = reshape(w(2:(1+param.nX*N)),param.nX,N);
utape = reshape(w(param.nX*N+2:end),param.nU,N);

%plot the trajectory in x1-x2 space
figure(2)
hold on
plot(xtape(1,:),xtape(2,:));
plot(0,0,'ro')
hold off

% plot the x1 state vs time
figure(3)
hold on
plot(xtape(1,:));
plot(0,0,'ro')
hold off

%plot the x2 state vs time
figure(4)
hold on
plot(xtape(2,:));
plot(0,0,'ro')
hold off

%plot the input trajectory vs time
figure(5)
plot(utape)

% draw the cartpole
figure(25)
for i=1:size(xtape,2)
    DRAW((i-1)*param.dt,xtape(:,i),param);
end

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% method sets up the optimization parameters
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function param=opt_param(param)

param.opt.x0=[0 0 0 0]'; %initial state
param.opt.xd=[0 pi 0 0]'; %final state
%bounds on final state
param.opt.xfmax=[0 pi 0 0]';
param.opt.xfmin=[0 pi 0 0]';

%bounds on state
param.opt.xmax=[10 10 10 10];
param.opt.xmin=[-10 -10 -10 -10];

```

```

%bounds on input
param.opt.umax=50;
param.opt.umax1=50;
param.opt.umin1=-50;
param.opt.umin=-50;

param.opt.nX=4; %state size
param.opt.nU=1; %input size
% costs for the optimization problem
param.opt.Q=eye(param.opt.nX)*10;
param.opt.Qf=eye(param.opt.nX);
param.opt.R=eye(param.opt.nU)*100;

%time step bounds
param.opt.hmin=param.dt;
param.opt.hmax=param.dt;

end

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Implements direct transcription with SNOPT
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function [w,F,info] = dirtran(N,DYNAMICS,GRADIENTS,param)
global SNOPT_USERFUN;

param.opt.xi=param.opt.x0;

nU=param.opt.nU;
nX=param.opt.nX;

%set up initial control input values
if isfield(param.opt,'u0s')
    u0=param.opt.u0s;
else
    u0 = zeros(nU*N,1);
end
% set up control input bounds
ulow = param.opt.umin*ones(nU*N,1);
uhigh = param.opt.umax*ones(nU*N,1);

x0=[];
xlow=[];
xhigh=[];

for k=1:nX
    x0=[x0;linspace(param.opt.x0(k),param.opt.xd(k),N)]; %initial state
    values
    xlow=[xlow;param.opt.xmin(k)*ones(1,N)]; %state upper bounds
    xhigh=[xhigh;param.opt.xmax(k)*ones(1,N)]; %state lower bounds
end

if isfield(param.opt,'x0s')
    x0=param.opt.x0s;

```

```

else

end

%time step bounds
hhigh=param.opt.hmax;
hlow=param.opt.hmin;

xlow(:,1)=param.opt.x0;
xhigh(:,1)=param.opt.x0;
xlow(:,end)=param.opt.xfmin;
xhigh(:,end)=param.opt.xfmax;

% create single vector of optimization problem parameters
w0 = [2/N;x0(:);u0]; %initial parameter values
wlow = [hlow;xlow(:);ulow]; %parameter upper bounds
whigh = [hhigh;xhigh(:);uhigh]; %parameter lower bounds

A = [];
iAfun = [];
jAvar = [];

% Cost function bounds
Flow = -inf;
Fhigh = inf;
iGfun = ones(length(w0),1);
jGvar = [1:length(w0)]';
% f is the vector of costs + constraint evaluations f=[J;c1;c2...]
% G is the sparse matrix of cost and constraint gradients
% iGfun and iGvar represent the indices in G that are non-zero
nf = 1;

% set up the sparse structure
for i=1:N-1,
    for j=1:nX,
        nf = nf+1;
        iGfun = [iGfun;nf*ones(1+2*nX+nU,1)];
        jGvar = [jGvar;1;(1:2*nX)'+(nX*i-(nX-1));(1:nU)'+(nX*N+1)+nU*(i-1)];
        Flow = [Flow;0];
        Fhigh = [Fhigh;0];
    end
end

% set up the SNOPT parameters
setSNOPTParam('Major Iterations Limit',10000);
setSNOPTParam('Minor Iterations Limit',500);
setSNOPTParam('Major Optimality Tolerance',1e-6);
setSNOPTParam('Major Feasibility Tolerance',1e-6);
setSNOPTParam('Minor Feasibility Tolerance',1e-6);
setSNOPTParam('Superbasics Limit',200);
setSNOPTParam('Derivative Option',0);
setSNOPTParam('Verify Level',0);
setSNOPTParam('Iterations Limit',10000);

```



```

A = [];
iAfun = [];
jAvar = [];

%define the SNOPT userfunction
SNOPT_USERFUN = @(w) userFun(w,N,DYNAMICS,GRADIENTS,param);

%set up inputs for SNOPT solver
wmul = zeros(length(w0), 1);
wstate = zeros(length(w0), 1);

Fmul = zeros(length(Flow), 1);
Fstate = zeros(length(Flow), 1);

options.name = 'Prob0';
options.start = 'Cold';
options.screen = 'on';
options.printfile = ['prob0-', num2str(get(getCurrentTask(),'ID')), '.out'
    ];

A_struct = struct('row', iAfun, 'col', jAvar, 'val', A);
G_struct = struct('row', iGfun, 'col', jGvar);
% call the SNOPT solver
[w, F, info] = snopt(w0, wlow, which, wmul, wstate, Flow, Fhigh, Fmul,
    Fstate, SNOPT_USERFUN, 0, 1, A_struct, G_struct, options);

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Function for evaluating the cost, constraints, and gradients
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function [f,G] = userFun(w,N,DYNAMICS,GRADIENTS,param)
persistent k;

if isempty(k)
    k=1;
end

nX=param.opt.nX;
nU=param.opt.nU;
I=eye(nX);

h = w(1);
X = reshape(w(2:(1+nX*N)),nX,N);
u = reshape(w(nX*N+2:end),nU,N);

f_dyn = [];
df_dyn = [];
f_u=[];
df_u=[];

J=0;
dJdx=[];
dJdu=[];

```

```

%construct the dynamics constraints and costs for each trajectory knot
%point
% f is the vector of costs + constraint evaluations f=[J;c1;c2...]
% G is the sparse matrix of cost and constraint gradients
for i=2:N

    t=0;

    xdot=DYNAMICS(t,X(:,i-1),u(:,i-1),param);

    f_dyn(:,i-1) = X(:,i) - X(:,i-1) - xdot*h; % the dynamics constraint

    [dfdx dfdu]=GRADIENTS(t,X(:,i-1),u(:,i-1),param);

    d_dyn=[-xdot -(eye(nX)+dfdx*h) eye(nX) -dfdu*h]; %the dynamics
        constraint gradients

    df_dyn(:, :, i-1) = d_dyn';

    [dJdxi dJdui]=cost_gradients(t,X(:,i-1),u(:,i-1),param); %cost
        gradients

    Ji=cost(t,X(:,i-1),u(:,i-1),param); %evaluate the cost function

    J=J+Ji; %define the cost

    dJdx=[dJdx dJdxi]; %define the cost gradient w.r.t. state

    dJdu=[dJdu dJdui]; %define th cost gradient w.r.t. input

end

[dJdxi, dJdui]=final_cost_gradients(t,X(:,N),u(:,N),param); %compute the
    final cost gradients

Ji=final_cost(t,X(:,N),u(:,N),param); %compute the final cost

J=J+Ji;

dJdx=[dJdx dJdxi];% append the final cost gradients w.r.t. state

dJdu=[dJdu dJdui];%append the final cost gradients w.r.t input

cost0 = J; % the total cost
dcost = [J/h dJdx dJdu]; %the cost gradients

f = [cost0;f_dyn(:)]; % cost and constraint evaluation
G = [dcost(:);df_dyn(:)]; % gradients

if param.plot==1

    plot(0,0, 'ko', 'MarkerSize',5, 'MarkerFaceColor',[0 0 0]);

```

```

    xlabel('X (m)', 'FontSize', 18);
    ylabel('Z (m)', 'FontSize', 18);
    h=gca;
    set(h, 'FontSize', 16);
    axis equal; axis([-4 1 -1 1.75]);
    hold off

end

k=k+1;

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Compute the final cost gradients
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function [dJdx dJdu]=final_cost_gradients(t,x,u,param)

Q=param.opt.Qf;
R=param.opt.R;

dJdx=2*x'*Q;
dJdu=0*2*u'*R;

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Compute the final cost
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function J=final_cost(t,x,u,param)

Q=param.opt.Qf;
R=param.opt.R;

J=x'*Q*x;

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Compute the running cost gradients
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function [dJdx dJdu]=cost_gradients(t,x,u,param)

Q=param.opt.Q;
R=param.opt.R;

dJdx=2*x'*Q;
dJdu=2*u'*R;

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Compute the running cost
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```
function J=cost(t,x,u,param)

Q=param.opt.Q;
R=param.opt.R;

J=x'*Q*x+u'*R*u;

end

function setSNOPTParam(paramstring,default)
    snset([paramstring,'=',num2str(default)]);
end
```